Exception Handling Part 2

Compile-Time Errors

- All syntax errors will be detected and displayed by the Java compiler.
- When the compiler displays an error, it will not create the .class file.

✓ The most common problems are:

✓ Missing semicolons

✓ Missing brackets

✓ Misspelling of identifiers and keywords

✓ Missing double quotes in strings

✓ Use of undeclared variables

- ✓ Incompatible types in assignments
- ✓ Bad reference to objects
- ✓ Use of = in place of == operator
- ✓ And so on

Runtime Error

- Sometimes a program may compile successfully creating the .class file but may not run properly.
- Such programs may produce wrong results due to wrong logic or may terminate due to errors. Those are called run time errors

✓ Most common run-time errors are:

✓ Dividing an integer by zero

✓ Accessing an element that is ou of the bounds of an array

- ✓ Trying to store a value into an array of an incompatible class or type.
- ✓ Trying to cast an instance of a class to one of its subclasses
- ✓ Passing a parameter that is not in a valid range or value for a method
- ✓ Trying to illegally change the state of a thread
- ✓ Attempting to use a negative size for an array

- ✓ Using a null object reference as a legitimate object reference to access a method or a variable.
- ✓ Converting invalid string to a number
- ✓ Accessing a character that is out of bounds of a string.
- ✓ And many more

- The call stack is quite useful for debugging, because it pinpoints the precise sequence of steps that led to the error.

throws

- If a method is capable of causing an exception that it does not handle, it must specify this behavior so that callers of the method can guard themselves against that exception.
- You do this by including a throws clause in the method's declaration.
- A throws clause lists the types of exceptions that a method might throw.
- This is necessary for all exceptions, except those of type Error or RuntimeException, or any of their subclasses.
- All other exceptions that a method can throw must be declared in the throws clause. If they are not, a compile-time error will result.

```
type method-name(parameter-list) throws exception-list
{
  // body of method
}
```

Method without throws

```
// This program contains an error and will not compile.
class ThrowsDemo {
  static void throwOne() {
    System.out.println("Inside throwOne.");
    throw new IllegalAccessException("demo");
  }
  public static void main(String args[]) {
    throwOne();
  }
}
```

Method with throws clause

```
// This is now correct.
class ThrowsDemo {
  static void throwOne() throws IllegalAccessException {
    System.out.println("Inside throwOne.");
    throw new IllegalAccessException("demo");
  }
  public static void main(String args[]) {
    try {
      throwOne();
    } catch (IllegalAccessException e) {
      System.out.println("Caught " + e);
    }
  }
}
```

Here is the output generated by running this example program:

```
inside throwOne
caught java.lang.IllegalAccessException: demo
```

finally block

- Resource leaks
  - Programs obtain and do not return resources
  - Automatic garbage collection avoids most memory leaks
    - Other leaks can still occur
- **`finally`** block
  - Placed after last **`catch`** block
  - Can be used to returns resources allocated in **`try`** block
  - Always executed, irregardless whether exceptions thrown or caught
  - If exception thrown in **`finally`** block, processed by enclosing **`try`** block
  - REMEMBER If a finally block is associated with a try, the finally block will be executed upon conclusion of the try.

```
1  // Fig. 14.9: UsingExceptions.java
2  // Demonstration of stack unwinding.
3  public class UsingExceptions {
4     public static void main( String args[] )
5     {
6        try {
7           throwException();
8        }
9        catch ( Exception e ) {
10          System.out.println( "Exception handled in main" );
11       }
12    }
13
14    public static void throwException() throws Exception
15    {
16       // Throw an exception and catch it in main.
17       try {
18          System.out.println( "Method throwException" );
19          throw new Exception();      // generate exception
20       }
21       catch( RuntimeException e ) {  // nothing caught here
22          System.err.println( "Exception handled in " +
23                              "method throwException" );
24       }
25       finally {
26          System.err.println( "Finally is always executed" );
27       }
28    }
29 }
```

```
Method throwException
Finally is always executed
Exception handled in main
```

Using `printStackTrace` and `getMessage`

- Class **`Throwable`**
  - Superclass of all exceptions
  - Offers method **`printStackTrace`**
    - Prints method call stack for caught **`Exception`** object
      - Most recent method on top of stack
    - Helpful for testing/debugging
  - Constructors
    - **`Exception()`**
    - **`Exception( String informationString )`**
  - **`informationString`** may be accessed with method **`getMessage`**

```
1  // Fig. 14.10: UsingExceptions.java
2  // Demonstrating the getMessage and printStackTrace
3  // methods inherited into all exception classes.
4  public class UsingExceptions {
5     public static void main( String args[] )
6     {
7        try {
8           method1();
9        }
10       catch ( Exception e ) {
11          System.err.println( e.getMessage() + "\n" );
12
13          e.printStackTrace();
14       }
15    }
16
17    public static void method1() throws Exception
18    {
19       method2();
20    }
21
22    public static void method2() throws Exception
23    {
24       method3();
25    }
26
27    public static void method3() throws Exception
28    {
29       throw new Exception( "Exception thrown in method3" );
30    }
31 }
```

Call **method1**, which calls **method2**, which calls **method3**, which throws an exception.

getMessage prints the **String** the **Exception** was initialized with.

printStackTrace prints the methods in this order:
method3
method2
method1
main
(order they were called when exception occurred)

Output:

```
Exception thrown in method3
java.lang.Exception: Exception thrown in method3
        at UsingExceptions.method3(UsingExceptions.java:28)
        at UsingExceptions.method2(UsingExceptions.java:23)
        at UsingExceptions.method1(UsingExceptions.java:18)
        at UsingExceptions.main(UsingExceptions.java:8)
```

Throwing our own exception

There may be times when we would like to throw our own exceptions. We can do this by using the keyword throw as follows:

Throw new Throwable_subclass

```
class MyException extends Exception
{
    MyException(String message)
    {
        super(message);
    }
}
public class Main {

    public static void main(String[] args) {

        int x =5, y =1000;
        try
        {
            float z = (float) x / (float) y;
            if(z < 0.01)
            {
                throw new MyException("Number is too small");
            }
        }
        catch(MyException e)
        {
            System.out.println("Caught my exception");
            System.out.println(e.getMessage());
        }
        finally
        {
            System.out.println("This will be executed");
        }
    }
}
```

Output:

```
Caught my exception
Number is too small
This will be executed
```

What is the output ?

```
public class Main {

    public static void main(String[] args) {
        try
        {
           int n = Integer.parseInt(args[0]);
           int n1 = Integer.parseInt(args[1]);
           int n2 = n1 + n;
        }
        catch(ArithmeticException e)
        {
            System.out.println("Arithmetic Exception block
1");
        }
        catch(ArithmeticException e)
        {
            System.out.println("Arithmetic Exception block
2");
        }
        finally
        {
            System.out.println("This will be executed");
        }
    }
}
```

Output:

```
java: exception
java.lang.ArithmeticException has already
been caught
```

What is the output?

```
public class Main {

    public static void main(String[] args) {

       try
        {
           int n = Integer.parseInt(args[0]);
           int n1 = Integer.parseInt(args[1]);
           int n2 = n1 + n;
        }
        catch(ArithmeticException e)
        {
            System.out.println("Arithmetic Exception");
        }
        catch(NumberFormatException e)
        {
            System.out.println("Number Format
Exception");
        }
        catch(Exception e )
        {
            System.out.println(e);
        }
    }
}
```

Java’s Built-in Exceptions

Inside the standard package java.lang, Java defines several exception classes. A few have been used by the preceding examples.

The most general of these exceptions are subclasses of the standard type RuntimeException.

As previously explained, these exceptions need not be included in any method’s throws list.

In the language of Java, these are called unchecked exceptions because the compiler does not check to see if a method handles or throws these exceptions.

The unchecked exceptions defined in java.lang are listed in the following table

Exception	Meaning
ArithmeticException	Arithmetic error, such as divide-by-zero.
ArrayIndexOutOfBoundsException	Array index is out-of-bounds.
ArrayStoreException	Assignment to an array element of an incompatible type.
ClassCastException	Invalid cast.
EnumConstantNotPresentException	An attempt is made to use an undefined enumeration value.
IllegalArgumentException	Illegal argument used to invoke a method.
IllegalCallerException	A method cannot be legally executed by the calling code.
IllegalMonitorStateException	Illegal monitor operation, such as waiting on an unlocked thread.
IllegalStateException	Environment or application is in incorrect state.
IllegalThreadStateException	Requested operation not compatible with current thread state.
IndexOutOfBoundsException	Some type of index is out-of-bounds.
LayerInstantiationException	A module layer cannot be created.
NegativeArraySizeException	Array created with a negative size.
NullPointerException	Invalid use of a null reference.
NumberFormatException	Invalid conversion of a string to a numeric format.
SecurityException	Attempt to violate security.
StringIndexOutOfBoundsException	Attempt to index outside the bounds of a string.
TypeNotPresentException	Type not found.
UnsupportedOperationException	An unsupported operation was encountered.

Following Table lists those exceptions defined by java.lang that must be included in a method's throws list if that method can generate one of these exceptions and does not handle it itself.

These are called **checked exceptions.**

In addition to the exceptions in java .lang, Java defines several more that relate to its other standard packages.

Checked Exceptions

Exception	Meaning
ClassNotFoundException	Class not found.
CloneNotSupportedException	Attempt to clone an object that does not implement the **Cloneable** interface.
IllegalAccessException	Access to a class is denied.
InstantiationException	Attempt to create an object of an abstract class or interface.
InterruptedException	One thread has been interrupted by another thread.
NoSuchFieldException	A requested field does not exist.
NoSuchMethodException	A requested method does not exist.
ReflectiveOperationException	Superclass of reflection-related exceptions.

Using checked exception IllegalAccessException

```
class ThrowsDemo {
  static void throwOne() throws IllegalAccessException {
    System.out.println("Inside throwOne.");
    throw new IllegalAccessException("demo");
  }
  public static void main(String args[]) {
    try {
      throwOne();
    } catch (IllegalAccessException e) {
      System.out.println("Caught " + e);
    }
  }
}
```

Exception constructors

Exception defines four public constructors.

Exception( )
Exception(String msg)
Throwable(Throwable causeExc)
Throwable(String msg, Throwable causeExc)

The version of toString( ) defined by Throwable (and inherited by Exception) first displays the name of the exception followed by a colon, which is then followed by your description. By overriding toString( ), you can prevent the exception name and colon from being displayed.

An example of overriding the toString() method

```
// This program creates a custom exception type.
class MyException extends Exception {
  private int detail;

  MyException(int a) {
    detail = a;
  }

  public String toString() {
    return "MyException[" + detail + "]";
  }
}

class ExceptionDemo {
  static void compute(int a) throws MyException {
    System.out.println("Called compute(" + a + ")");
    if(a > 10)
      throw new MyException(a);
    System.out.println("Normal exit");
  }

  public static void main(String args[]) {
    try {
      compute(1);
      compute(20);
    } catch (MyException e) {
      System.out.println("Caught " + e);
    }
  }
}
```

This example defines a subclass of Exception called MyException.

This subclass is quite simple: It has only a constructor plus an overridden toString( ) method that displays the value of the exception.

The ExceptionDemo class defines a method named compute( ) that throws a MyException object.

The exception is thrown when compute( )'s integer parameter is greater than 10.

The main( ) method sets up an exception handler for MyException, then calls compute( ) with a legal value (less than 10) and an illegal one to show both paths through the code.

Output:

```
Called compute(1)
Normal exit
Called compute(20)
Caught MyException[20]
```